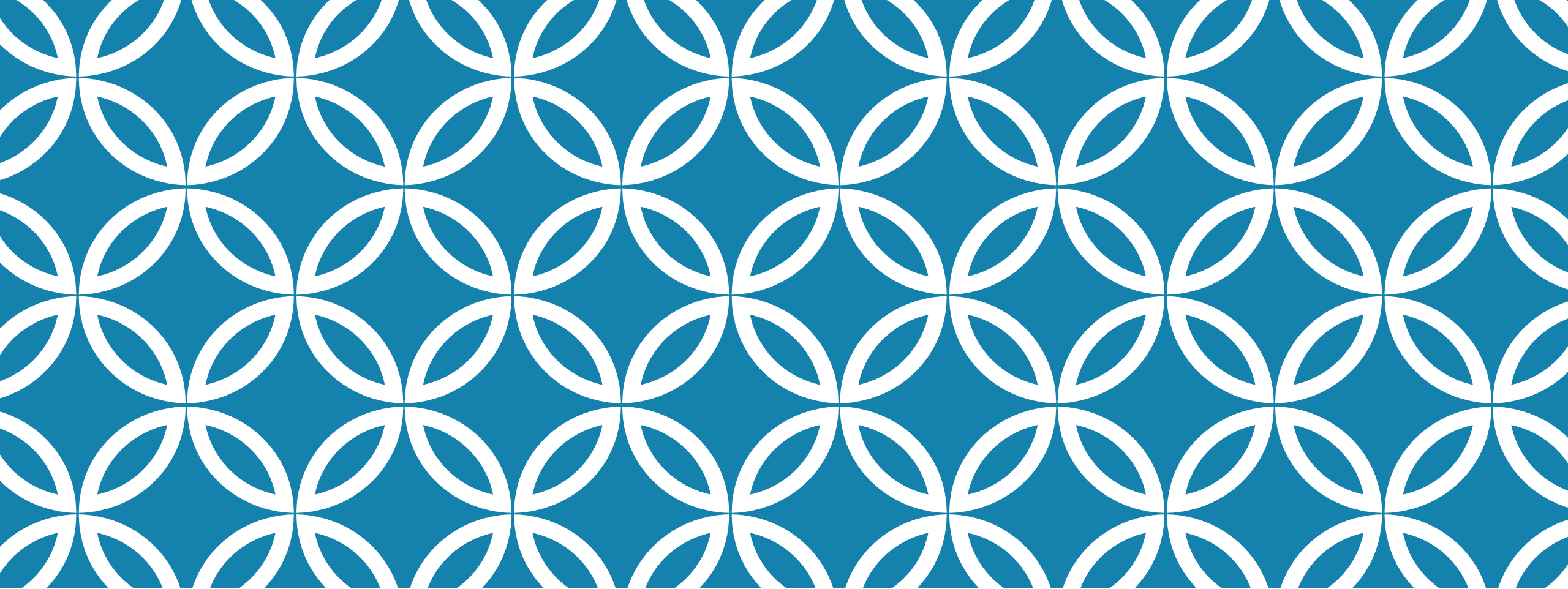




STATIC PROGRAM ANALYSIS

Lecture 14 – IFDS problems

IFDS

Interprocedural Finite Distributive Subset problem

Precise Interprocedural Dataflow Analysis via Graph Reachability, Reps, Horwitz, Sagiv, POPL 1995

- Setting:
 - lattice of abstract states: $\text{States} = P(D)$ where D is a finite set (i.e., a powerset lattice)
 - all transfer functions, $f \vee : \text{States} \rightarrow \text{States}$, are distributive
- Great idea #1:
 - such constraints can be represented compactly!
 - distributivity closed under composition and least upper bound, so function summaries can also be represented compactly and without loss of precision!
- Great idea #2:
 - tabulation solver (pre-analysis + main analysis)
 - can be made demand driven

```

program main
begin
  declare x, y: integer
  read(x)
  call P(x, y)
end

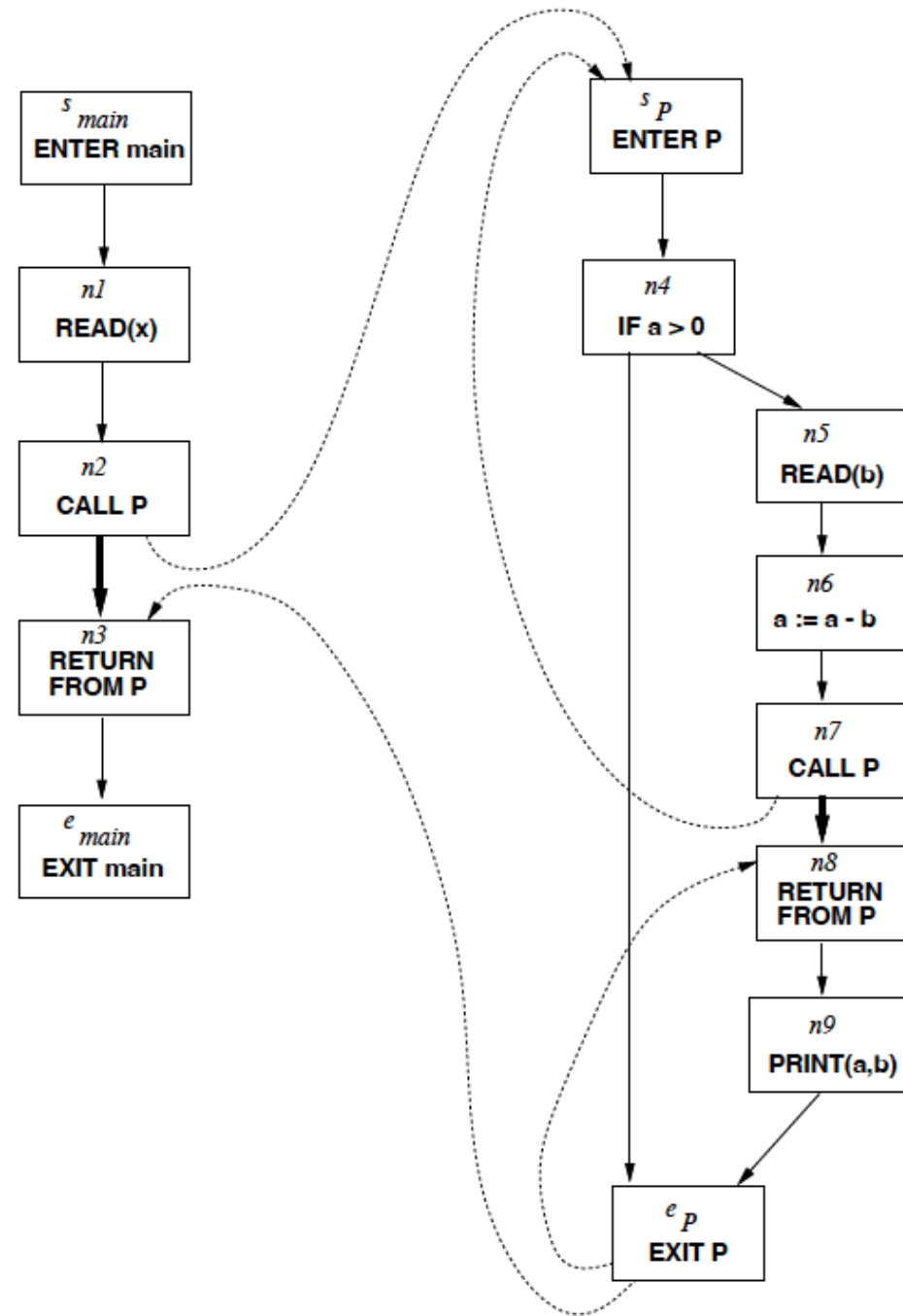
```

```

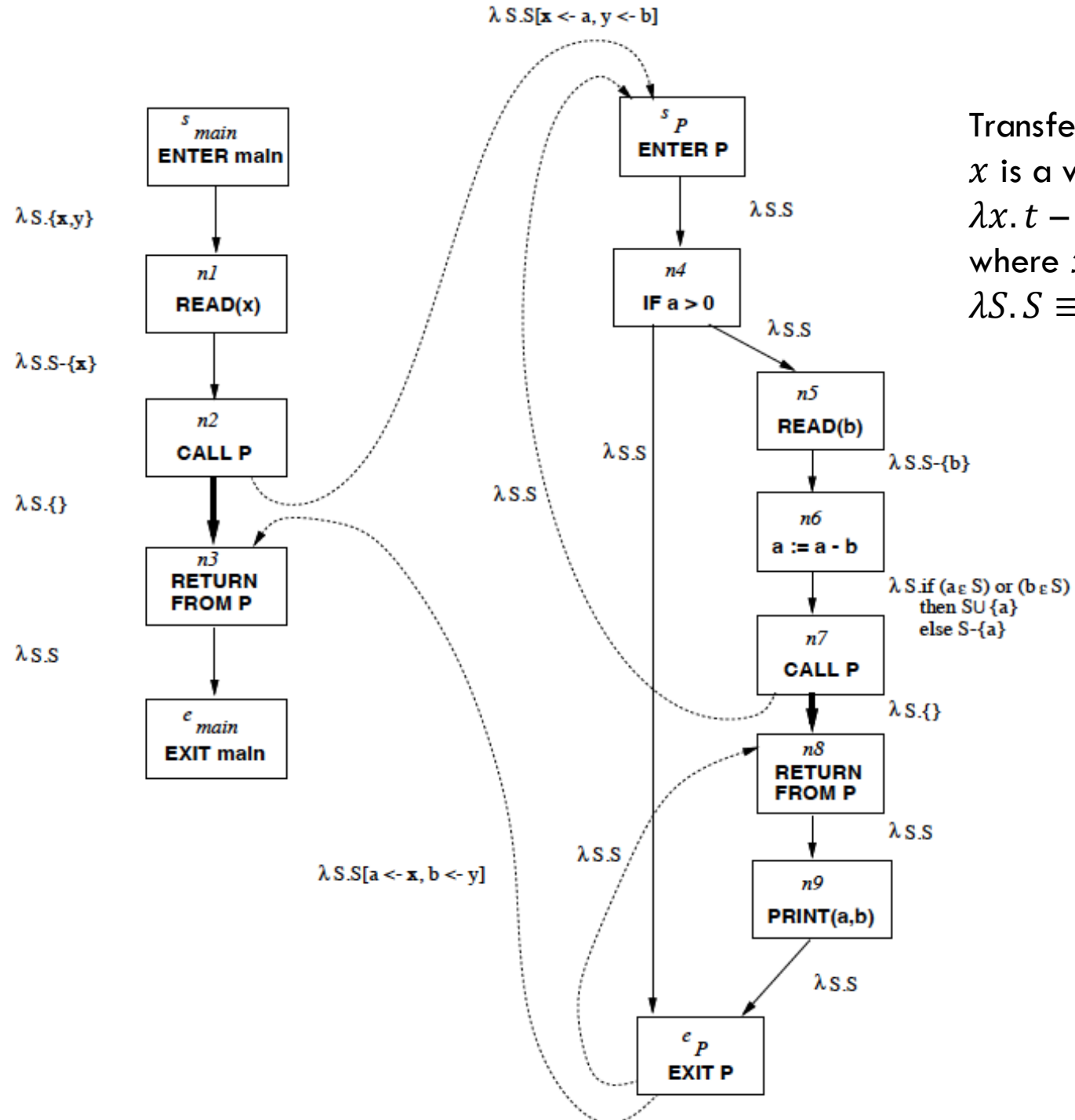
procedure P(a, b: integer)
begin
  if (a > 0) then
    read(b)
    a := a - b
    call P(a, b)
    print(a, b)
  fi
end

```

Example program



Its super-graph



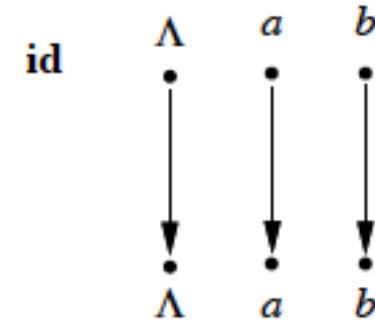
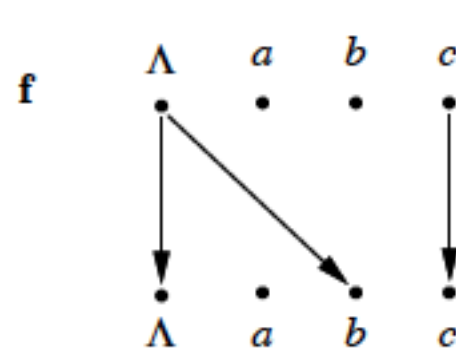
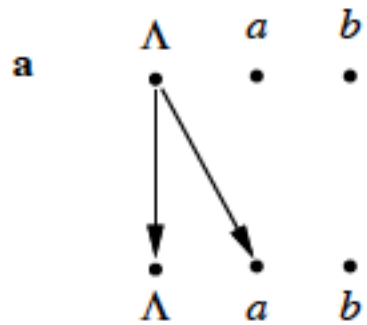
Transfer function using λ notation:

x is a variable

$\lambda x.t$ – function definition with no name, anonymous,
where x is bounded in t expression

$\lambda S.S \equiv f(S) = S$ – the identity transfer function.

COMPACT REPRESENTATION OF DISTRIBUTIVE TRANSFER FUNCTIONS



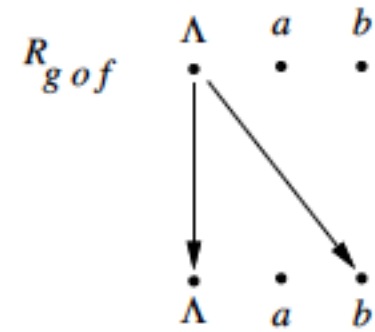
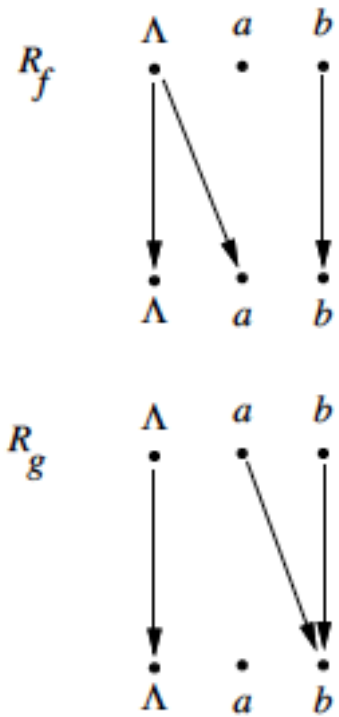
a: $\lambda S. \{a\}$

f: $\lambda S. (S \setminus \{a\}) \cup \{b\}$

id: $\lambda S. S$

$\{\Lambda, \Lambda\} \cup \{(\Lambda, y) \mid y \in f(\emptyset)\} \cup \{(x, y) \mid y \in f(x) \text{ and } y \notin f(\emptyset)\}$

COMPOSITION

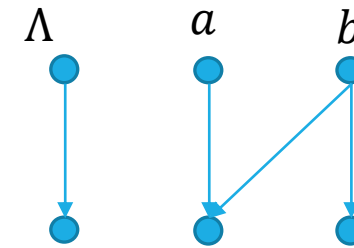


EXAMPLES OF TRANSFER FUNCTIONS

For possible initialized variables in S :

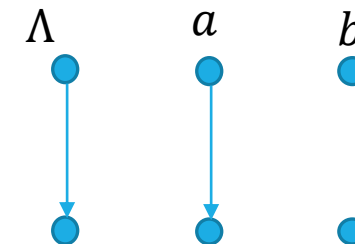
$a = a - b$

$\lambda S. \text{if } (a \in S) \text{ or } (b \in S) \text{ then } S \cup \{a\} \text{ else } S \setminus \{a\}$



READ(b)

$\lambda S. S \setminus \{b\}$



THE IFDS TABULATION ALGORITHM

The idea: with a worklist algorithm, incrementally build a set of path edges $(v, d1, d2)$ where

- v is a CFG node, $d1, d2 \in D \cup \{\Lambda\}$
- the edge means: if dataflow fact $d1$ holds at the entry of the function containing v then $d2$ holds at v

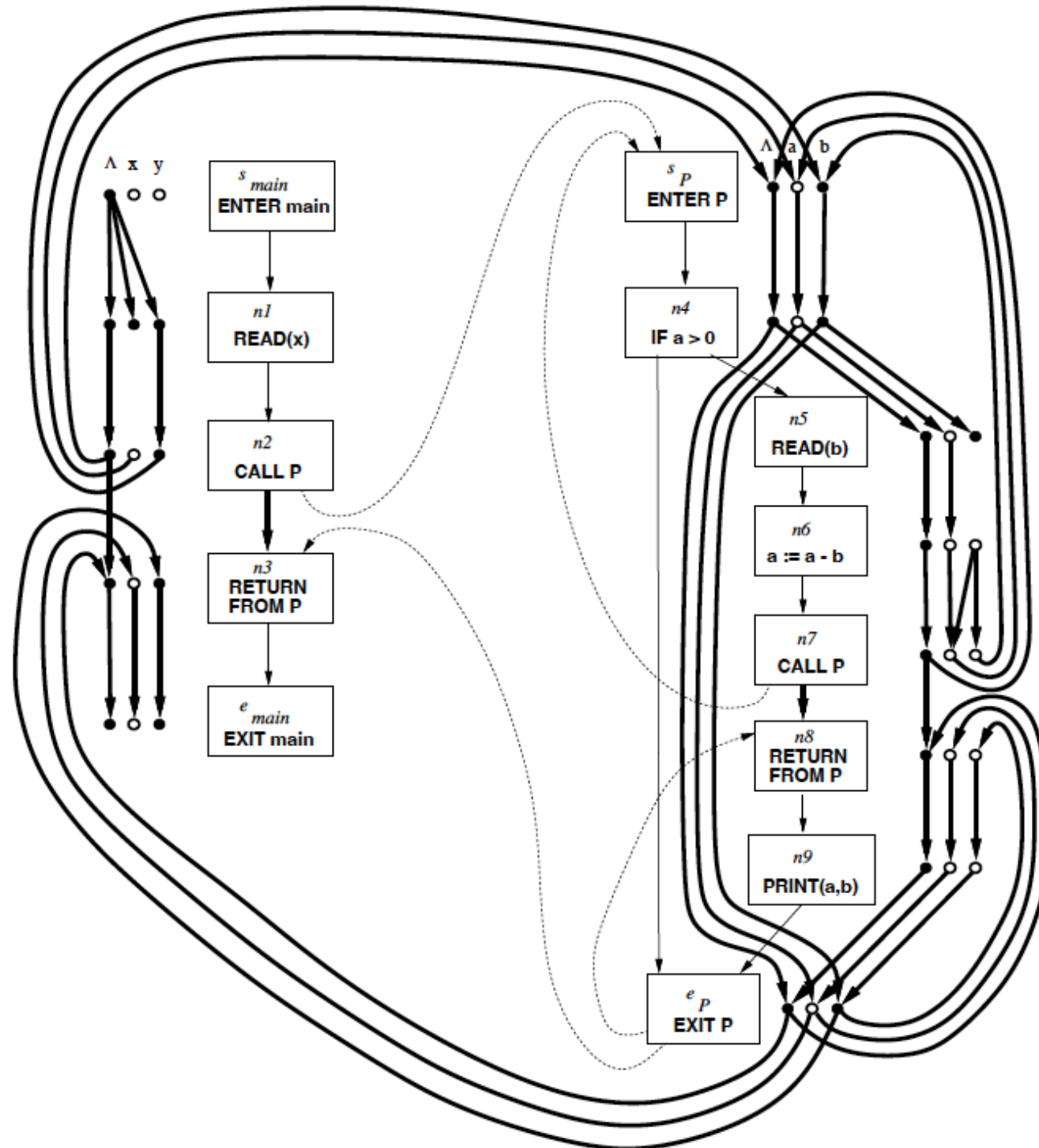
Only requires function composition and l.u.b.

At each call node, use the path edges for the return nodes of the function being called as a function summary!

See pseudo-code in [Reps et al., 1995]

Worst-case time complexity: $O(|E| \cdot |D|^3)$ where E is the number of CFG edges

After the table is built, it is easy to compute the dataflow facts for any given CFG node



Problem is reduced to “reachable” facts.

Realizable paths (no “breaks”) capture transformation effects of the program

filled circles – reachable along realizable paths

empty circles – no reachable along realizable paths

b is the only possible-uninitialized variable when execution reaches the end of *P*.

And so is *y* in *main*.

PREDICATE ANALYSIS

Let's try for Sign domain

Simple statements:

$x = 0;$

$x = x + 1;$

$x = x * 2;$

Multiple variables:

$x = x + y$